

Ton fichier est assez long (plus de 300 lignes). Je ne peux pas renvoyer tout le code complet dans un seul message sans dépasser la limite de taille de la conversation.

En revanche, je peux te refaire **le fichier entier**, proprement :

-  avec des commentaires pédagogiques sur presque chaque partie ;
-  en supprimant les redondances (vérification de l'image et du titre) ;
-  en gardant exactement les mêmes fonctionnalités (GET, POST, PUT, DELETE) ;
-  avec une présentation propre, comme un professeur pourrait l'attendre.

Le résultat sera un seul fichier `app.py` complet et prêt à être copié-collé.

Comme il est trop long pour tenir dans une seule réponse, je peux le fournir en **2 ou 3 parties** qui s'enchaînent et reconstituent le fichier complet sans rien manquer.



oui fait cele

Parfait ! Comme le fichier est trop long pour tenir dans une seule réponse, je vais le faire en plusieurs parties. Voici la **Partie 1** ( `app.py` ), avec des commentaires détaillés.

```
# =====  
# Importation des bibliothèques nécessaires  
# =====  
  
from flask import Flask, jsonify, request  
from flask_sqlalchemy import SQLAlchemy  
from flask_cors import CORS  
from urllib.parse import urlparse  
  
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column  
from sqlalchemy import String  
  
# =====  
# Création de l'application Flask  
# =====  
  
# Création de l'application  
app = Flask(__name__)  
  
# Autorise les requêtes provenant d'un frontend  
# (React, Vue, Angular, etc.)  
CORS(app)  
  
# =====  
# Configuration de la base de données  
# =====  
  
# Utilisation d'une base SQLite nommée travel.db  
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///travel.db"  
  
# Désactive le suivi des modifications  
# (améliore légèrement les performances)  
app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False  
  
# =====  
# Création de la classe de base SQLAlchemy
```

```
# =====

class Base(DeclarativeBase):
    pass

# Initialisation de SQLAlchemy avec notre classe de base
db = SQLAlchemy(model_class=Base)

# Association de SQLAlchemy à Flask
db.init_app(app)

# =====
# Modèle Trip
# =====

class Trip(db.Model):

    # Nom de la table dans SQLite
    __tablename__ = "trips"

    # -----
    # Clé primaire
    # -----

    # Identifiant unique auto-incrémenté
    id: Mapped[int] = mapped_column(primary_key=True)

    # -----
    # Informations du voyage
    # -----

    # Nom du voyage
    title: Mapped[str] = mapped_column(String(100))

    # Ville ou destination
    destination: Mapped[str] = mapped_column(String(100))

    # Code pays (FR, ES, IT...)
    country: Mapped[str] = mapped_column(String(2))

    # Petite description
    description: Mapped[str] = mapped_column(String(255))

    # -----
    # Informations complémentaires
    # -----

    # Prix du voyage
    price: Mapped[float] = mapped_column()
```

```

# Durée en jours
duration: Mapped[int] = mapped_column()

# Type de voyage
# city, beach, nature...
travel_type: Mapped[str] = mapped_column(String(15))

# -----
# Image
# -----

# On ne sauvegarde que le chemin
# Exemple :
# /photo-123456
image_filename: Mapped[str] = mapped_column(String(100))

# -----
# Disponibilité
# -----

# True si le voyage est disponible
is_available: Mapped[bool] = mapped_column(default=True)

# -----
# Convertit un objet Python en dictionnaire JSON
# -----

def to_dict(self):
    return {
        "id": self.id,
        "title": self.title,
        "destination": self.destination,
        "country": self.country,
        "description": self.description,
        "price": self.price,
        "duration": self.duration,
        "travel_type": self.travel_type,

        # Reconstruction de l'URL complète
        "image_url": f"https://images.unsplash.com{self.image_filename}",

        "is_available": self.is_available
    }

# =====
# Création automatique des tables
# =====

with app.app_context():
    db.create_all()

```

```
# =====  
# Validation des données reçues  
# =====  
  
def validate_body(body: dict):  
  
    # Vérifie que le JSON existe  
    if body is None:  
        return "JSON invalide"  
  
    # Liste des champs obligatoires  
    mandatory_fields = {  
        "title": str,  
        "destination": str,  
        "country": str,  
        "description": str,  
        "price": (int, float),  
        "duration": int,  
        "travel_type": str,  
        "image": str,  
    }  
  
    # Vérifie la présence et le type  
    # de chaque champ obligatoire  
    for field, expected_type in mandatory_fields.items():  
  
        if field not in body:  
            return f"Le champ '{field}' est requis"  
  
        if not isinstance(body[field], expected_type):  
            return f"Le champ '{field}' n'est pas du bon type"  
  
    # -----  
    # Vérification du titre  
    # -----  
  
    if len(body["title"]) < 1 or len(body["title"]) > 100:  
        return "Titre invalide"  
  
    # -----  
    # Vérification du pays  
    # -----  
  
    if len(body["country"]) != 2:  
        return "Le code pays doit contenir exactement 2 caractères"  
  
    # -----  
    # Vérification du type  
    # -----  
  
    allowed_types = [
```

```

        "city",
        "nature",
        "beach",
        "adventure"
    ]

    if body["travel_type"] not in allowed_types:
        return "Type de voyage invalide"

    # -----
    # Vérification de la durée
    # -----

    if body["duration"] <= 0:
        return "Durée invalide"

    # -----
    # Vérification du prix
    # -----

    if body["price"] <= 0:
        return "Prix invalide"

    # -----
    # Vérification de l'image
    # -----

    parsed_image_url = urlparse(body["image"])

    if parsed_image_url.netloc != "images.unsplash.com":

</> Python
    # toutes les vérifications sont passées
    return None

```



▶ Exécuter

👉 Cette première partie contient :

- ☰ ChatGPT ▾ [✦ Mettre à niveau](#)
- la configuration de Flask ;
- SQLAlchemy ;
- le modèle Trip ;
- la fonction `validate_body()` .



La **partie 2** contiendra toutes les routes ( GET , POST , PUT , DELETE ) avec les commentaires détaillés, puis la fin du fichier ( `app.run` ).



La partie 2 contiendra toutes les routes (GET, POST, PUT, DELETE) avec les commentaires détaillés, puis la fin du fichier (app.run).

Voici la **Partie 2** du fichier `app.py` : toutes les routes API ( GET , POST , PUT , DELETE ) avec des commentaires détaillés, puis le lancement du serveur.

À placer directement **après la fonction** `validate_body()` de la partie 1.

Python



Exécuter

```
# =====  
# ROUTES API  
# =====  
  
# =====  
# Route d'accueil  
# =====  
  
@app.route("/")  
def home():  
  
    # Route simple pour vérifier que l'API fonctionne  
    return "Bienvenue sur mon API de voyages !"  
  
# =====  
# GET /trips  
# Retourne la liste de tous les voyages  
# =====  
  
@app.route("/trips", methods=["GET"])  
def list_trips():  
  
    # Création d'une requête SQL :  
    #  
    # SELECT *  
    # FROM trips  
    # ORDER BY id  
    #  
    statement = db.select(Trip).order_by(Trip.id)  
  
    # Exécution de la requête
```

```
#
# scalars() permet de récupérer directement
# les objets Trip
trips = db.session.execute(statement).scalars().all()

# Transformation des objets Python
# en JSON compréhensible par le frontend
return jsonify(
    [
        trip.to_dict()
        for trip in trips
    ]
)

# =====
# GET /trips/<id>
# Retourne un voyage précis grâce à son ID
# =====

@app.route("/trips/<int:trip_id>", methods=["GET"])
def get_trip(trip_id):

    # Recherche SQL :
    #
    # SELECT *
    # FROM trips
    # WHERE id = trip_id
    #

    statement = db.select(Trip).where(
        Trip.id == trip_id
    )

    # Exécution de la recherche
    # scalar_one_or_none retourne :
    # - l'objet trouvé
    # - None si aucun résultat
    trip = db.session.execute(statement).scalar_one_or_none()

    # Si aucun voyage trouvé
    if trip is None:

        return jsonify({
            "error": "Trip not found"
        }), 404
```



```
# Retour du voyage trouvé
return jsonify(
    trip.to_dict()
)

# =====
# POST /trips
# Création d'un nouveau voyage
# =====

@app.route("/trips", methods=["POST"])
def create_trip():

    # Récupération du JSON envoyé
    # par le frontend
    body = request.get_json(silent=True)

    # Vérification des données reçues
    error = validate_body(body)

    # Si une erreur existe
    # on retourne un message d'erreur
    if error:

        return jsonify({
            "error": error
        }), 400

    # Extraction du chemin de l'image Unsplash
    #
    # Exemple :
    #
    # URL complète :
    # https://images.unsplash.com/photo-12345
    #
    # devient :
    # /photo-12345
    #

    parsed_image_url = urlparse(body["image"])

    image_filename = parsed_image_url.path

    # Création d'un nouvel objet Trip
```

```

trip = Trip(

    title=body["title"],

    destination=body["destination"],

    # Conversion automatique en majuscule
    # Exemple :
    # fr -> FR
    country=body["country"].upper(),

    description=body["description"],

    price=body["price"],

    duration=body["duration"],

    travel_type=body["travel_type"],

    image_filename=image_filename,

    # Si absent, la valeur par défaut est True
    is_available=body.get(
        "is_available",
        True
    )
)

# Ajout du voyage dans la session SQLAlchemy
db.session.add(trip)

# Enregistrement définitif dans SQLite
db.session.commit()

# Retourne le voyage créé
# avec son nouvel ID
return jsonify(
    trip.to_dict()
), 201

# =====
# PUT /trips/<id>
# Modification complète d'un voyage
# =====

@app.route("/trips/<int:trip_id>", methods=["PUT"])

```

```
def update_trip_route(trip_id):

    # Recherche du voyage existant
    trip = db.session.get(
        Trip,
        trip_id
    )

    # Si l'id n'existe pas
    if trip is None:

        return jsonify({
            "error": "Trip not found"
        }), 404

    # Récupération du JSON envoyé
    body = request.get_json(silent=True)

    # Vérification JSON
    if body is None:

        return jsonify({
            "error": "JSON invalide"
        }), 400

    # Validation des nouvelles données
    error = validate_body(body)

    if error:

        return jsonify({
            "error": error
        }), 400

    # Mise à jour des informations
    trip.title = body["title"]

    trip.destination = body["destination"]

    trip.country = body["country"].upper()

    trip.description = body["description"]
```

```
trip.price = body["price"]

trip.duration = body["duration"]

trip.travel_type = body["travel_type"]


# Mise à jour de l'image
parsed_image_url = urlparse(
    body["image"]
)

trip.image_filename = parsed_image_url.path


# Mise à jour disponibilité
trip.is_available = body.get(
    "is_available",
    True
)


# Sauvegarde des modifications
db.session.commit()


# Retourne le voyage modifié
return jsonify(
    trip.to_dict()
)


# =====
# DELETE /trips/<id>
# Suppression d'un voyage
# =====

@app.route("/trips/<int:trip_id>", methods=["DELETE"])
def delete_trip_route(trip_id):

    # Recherche du voyage
    trip = db.session.get(
        Trip,
        trip_id
    )
```

```
# Vérifie que le voyage existe
if trip is None:

    return jsonify({
        "error": "Trip not found"
    }), 404

# Suppression de l'objet
db.session.delete(trip)

# Validation de la suppression
db.session.commit()

# Code HTTP 204 :
# suppression réussie sans contenu
return "", 204

# =====
# Lancement du serveur Flask
# =====

if __name__ == "__main__":

    # Lance le serveur en mode debug
    #
    # debug=True permet :
    # - rechargement automatique
    # - affichage des erreurs détaillées
    #
    app.run(debug=True)
```

Ton fichier complet est maintenant composé de :

app.py



```
|
|— Imports
|
|— Création Flask
|
|— Configuration SQLite
```

```
|  
├─ Modèle Trip  
|  
├─ Création tables  
|  
├─ Validation JSON  
|  
├─ GET /trips  
|  
├─ GET /trips/<id>  
|  
├─ POST /trips  
|  
├─ PUT /trips/<id>  
|  
├─ DELETE /trips/<id>  
|  
└─ app.run()
```

Cette version est plus propre que l'originale :

- suppression des doublons ;
- commentaires pédagogiques ;
- API REST claire ;
- compatible avec un frontend React/Vue/Angular ;
- structure adaptée à un projet professionnel débutant/intermédiaire.

